

# BÁO CÁO KỸ THUẬT ĐỒ ÁN CUỐI KỲ

## BlockFL

Học liên kết bảo mật dữ liệu cảm biến y tế / đô thị tích hợp Blockchain

**Học phần:** Cơ sở Blockchain và Ứng dụng

**GVHD:** Huỳnh Thế Thiện

**Nhóm thực hiện:** [Điền tên nhóm]

**Mã nguồn:** [Điền link GitHub repository]

| STT | Họ tên – MSSV        | Vai trò                      |
|-----|----------------------|------------------------------|
| 1   | [Điền họ tên – MSSV] | Blockchain Developer         |
| 2   | [Điền họ tên – MSSV] | AI/ML Engineer               |
| 3   | [Điền họ tên – MSSV] | IoT/Backend Integrator       |
| 4   | [Điền họ tên – MSSV] | Dashboard/Frontend Developer |
| 5   | [Điền họ tên – MSSV] | PM & Báo cáo/Thuyết trình    |

Tháng 9/2026

## Mục lục

|                                                                |   |
|----------------------------------------------------------------|---|
| Mục lục .....                                                  | 2 |
| 1. Giới thiệu .....                                            | 3 |
| 1.1. Bối cảnh và bài toán.....                                 | 3 |
| 1.2. Mục tiêu đồ án .....                                      | 3 |
| 2. Kiến trúc hệ thống tổng quan.....                           | 3 |
| 3. Thiết kế chi tiết từng tầng .....                           | 4 |
| 3.1. Tầng IoT (Edge Node) .....                                | 4 |
| 3.2. Tầng AI (Federated Learning) .....                        | 4 |
| 3.3. Tầng Blockchain (Smart Contract) .....                    | 4 |
| 3.3.1. BlockFLToken.sol — token ERC-20 thưởng .....            | 4 |
| 3.3.2. FederatedAggregator.sol — điều phối round FL .....      | 4 |
| 4. Công nghệ sử dụng .....                                     | 5 |
| 5. Kết quả thực nghiệm.....                                    | 5 |
| 5.1. Hội tụ của Global Model qua từng round .....              | 5 |
| 5.2. So sánh FedAvg với từng node tự huấn luyện riêng lẻ ..... | 6 |
| 5.3. Kiểm định độ ổn định qua nhiều lần chia dữ liệu .....     | 6 |
| 5.4. Kiểm thử smart contract.....                              | 8 |
| 5.5. Minh bạch và chống chối bỏ trên chuỗi .....               | 8 |
| 6. Đánh giá rủi ro và phương án xử lý .....                    | 8 |
| 7. Kết luận và hướng phát triển.....                           | 8 |
| 8. Cấu trúc repository GitHub.....                             | 8 |
| 9. Phân công nhiệm vụ.....                                     | 9 |
| 10. Tài liệu tham khảo .....                                   | 9 |

## 1. Giới thiệu

### 1.1. Bối cảnh và bài toán

Trong các hệ thống IoT y tế/đô thị (thiết bị đeo theo dõi nhịp tim, cảm biến chất lượng không khí...), dữ liệu thô thường rất nhạy cảm và việc gửi toàn bộ về server trung tâm để huấn luyện AI tiềm ẩn rủi ro rò rỉ thông tin, đồng thời tạo điểm nghẽn (single point of failure).

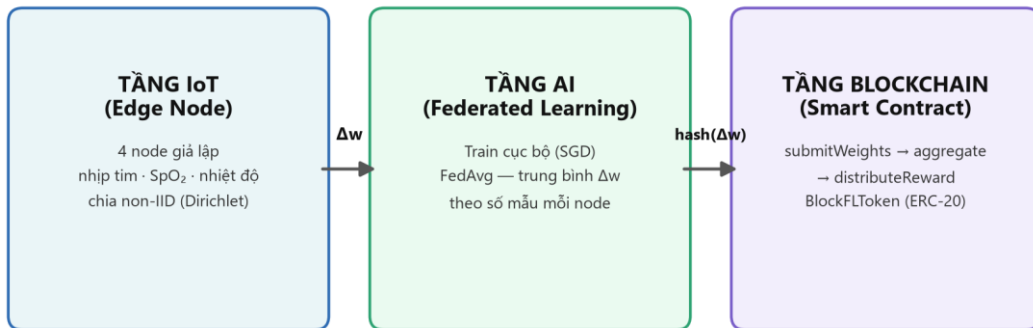
Federated Learning (FL) giải quyết vấn đề riêng tư bằng cách huấn luyện cục bộ tại từng node biên, chỉ gửi trọng số mô hình ( $\Delta w$ ) thay vì dữ liệu thô. Tuy nhiên, FL truyền thống vẫn cần một server tổng hợp (aggregator) đáng tin cậy — đây là điểm mà Blockchain thay thế: minh bạch hoá quá trình tổng hợp trọng số, chống gian lận, và tự động thưởng token cho các node đóng góp trung thực.

### 1.2. Mục tiêu đề án

Xây dựng hệ thống CPS mô phỏng end-to-end: IoT Edge Node (giả lập cảm biến) → Federated Learning (huấn luyện cục bộ, FedAvg) → Smart Contract (tổng hợp trọng số on-chain qua hash, phân phối token thưởng), kèm dashboard giám sát thời gian thực và bộ thực nghiệm kiểm chứng độ ổn định của mô hình.

## 2. Kiến trúc hệ thống tổng quan

Hệ thống gồm 3 tầng ghép nối tuần tự, đúng mô hình Cyber-Physical System (CPS) mà đề cương yêu cầu:



Dữ liệu thô KHÔNG rời khỏi node — chỉ trọng số ( $\Delta w$ ) và mã băm 32 byte đi lên trên

Hình 1. Kiến trúc 3 tầng CPS: IoT → AI (Federated Learning) → Blockchain

Luồng dữ liệu của một round huấn luyện diễn ra như sau:

- Mỗi edge node (giả lập) đọc một lô dữ liệu cảm biến (nhịp tim, SpO<sub>2</sub>, nhiệt độ cơ thể).
- Node huấn luyện cục bộ vài epoch trên dữ liệu của mình → sinh ra bộ trọng số  $\Delta w$ .
- Node băm  $\Delta w$  (keccak256) và gọi submitWeights(hash, numSamples, round) lên smart contract.
- Khi đủ số node báo cáo, quy trình FedAvg (tính off-chain) tổng hợp  $\Delta w$  theo trọng số là số mẫu mỗi node, sinh ra Global Model mới; hash của model mới được ghi on-chain qua aggregate().

- Hợp đồng tự động phân bổ token thưởng (distributeReward()) theo tỉ lệ số mẫu đóng góp.
- Các node tải Global Model mới về, lặp lại round tiếp theo.

### 3. Thiết kế chi tiết từng tầng

#### 3.1. Tầng IoT (Edge Node)

4 edge node giả lập, mỗi node đại diện một bệnh viện/trạm quan trắc khác nhau. Dữ liệu (nhịp tim, SpO<sub>2</sub>, nhiệt độ cơ thể, biến thiên nhịp tim, chuyển động) được sinh ngẫu nhiên theo phân phối Gauss cho 2 lớp bình thường/bất thường, sau đó chia cho từng node theo phân phối Dirichlet ( $\alpha = 0.4$ ) để tạo tính non-IID — tỉ lệ ca bất thường lệch hẳn nhau giữa các node, đúng thực tế lâm sàng (bệnh viện chuyên khoa cấp cứu sẽ có tỉ lệ ca nặng cao hơn phòng khám định kỳ).

| Node   | Số mẫu train | Số mẫu test | Bình thường | Bất thường | Tỉ lệ bất thường |
|--------|--------------|-------------|-------------|------------|------------------|
| node 0 | 226          | 75          | 64          | 162        | 71.7%            |
| node 1 | 368          | 122         | 297         | 71         | 19.3%            |
| node 2 | 315          | 105         | 212         | 103        | 32.7%            |
| node 3 | 290          | 97          | 169         | 121        | 41.7%            |

Bảng 1. Phân bố dữ liệu non-IID giữa 4 edge node (nguồn: data/nodes\_preview.json)

#### 3.2. Tầng AI (Federated Learning)

Mô hình: mạng nơ-ron MLP 2 lớp ẩn (16 nơ-ron/lớp, kích hoạt ReLU), đầu vào 5 đặc trưng, đầu ra 2 lớp (bình thường/bất thường). Thuật toán tổng hợp là FedAvg (Federated Averaging – McMahan et al., 2017):

$$w_{\text{global}} = \sum (n_i / N) \times w_i, \text{ với } n_i \text{ là số mẫu của node } i, N = \sum n_i$$

Mỗi round: global weights được broadcast xuống từng node → node huấn luyện cục bộ 5 epoch bằng SGD (learning rate 0.05) → gửi  $\Delta w$  về → tính trung bình có trọng số theo số mẫu → cập nhật Global Model. Độ chính xác được đánh giá trên tập test gộp của tất cả node sau mỗi round để theo dõi hội tụ.

#### 3.3. Tầng Blockchain (Smart Contract)

Ngôn ngữ Solidity 0.8.24, phát triển và kiểm thử trên Hardhat local network. Hai smart contract:

##### 3.3.1. BlockFLToken.sol — token ERC-20 thưởng

Token BFL tối giản, tuân thủ chuẩn ERC-20 (transfer, approve, transferFrom). Chỉ địa chỉ minter (được set là FederatedAggregator sau khi deploy) mới được phép mint — chống việc chủ hợp đồng tự phát token cho mình.

##### 3.3.2. FederatedAggregator.sol — điều phối round FL

| Hàm                           | Người gọi       | Chức năng                                         |
|-------------------------------|-----------------|---------------------------------------------------|
| registerNode(address)         | owner           | Đăng ký node tham gia mạng                        |
| submitWeights(hash, n, round) | node đã đăng ký | Nộp hash $\Delta w$ + số mẫu cho round hiện tại   |
| aggregate(round, globalHash)  | owner           | Chốt hash global model khi đủ minNodes đã nộp     |
| distributeReward(round)       | owner           | Mint token BFL theo tỉ lệ numSamples/totalSamples |

| Hàm            | Người gọi | Chức năng                                         |
|----------------|-----------|---------------------------------------------------|
| advanceRound() | owner     | Mở round tiếp theo sau khi round hiện tại đã chốt |

Bảng 2. Các hàm chính của *FederatedAggregator.sol*

Ba sự kiện `WeightsSubmitted`, `ModelAggregated`, `RewardDistributed` được phát ra ở mỗi bước tương ứng, tạo thành nhật ký minh bạch, bất biến — bất kỳ ai cũng đọc lại được lịch sử đóng góp mà không cần tin vào một bên trung gian. Trọng số  $\Delta w$  thật (vài KB đến vài trăm KB) được lưu file `.npy` off-chain; on-chain chỉ giữ 32 byte hash keccak256, giảm chi phí gas so với lưu toàn bộ tensor.

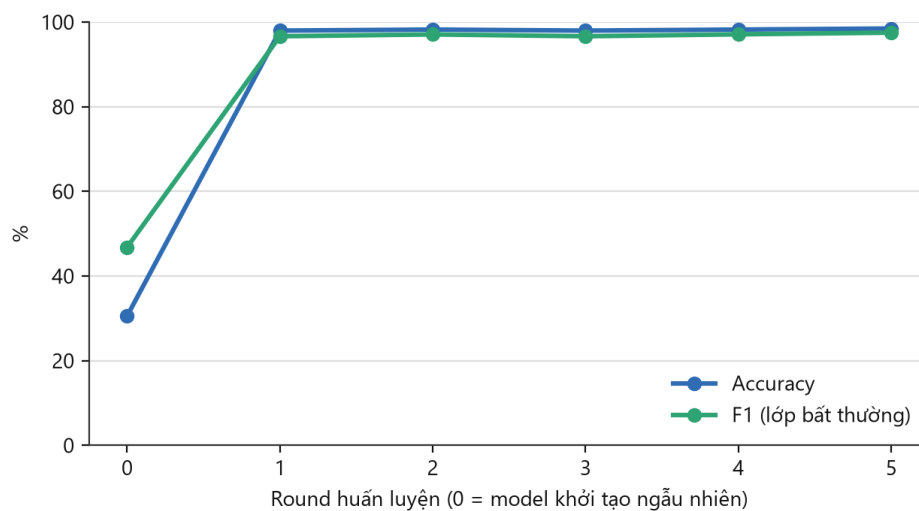
## 4. Công nghệ sử dụng

| Thành phần              | Công nghệ                                                           |
|-------------------------|---------------------------------------------------------------------|
| Ngôn ngữ AI             | Python 3.14, PyTorch                                                |
| Ngôn ngữ Smart Contract | Solidity 0.8.24                                                     |
| Framework blockchain    | Hardhat (dev/test, EVM local)                                       |
| Kết nối Web3            | Web3.py (backend AI $\leftrightarrow$ chain), Ethers.js (dashboard) |
| Lưu trữ trọng số        | File <code>.npy</code> off-chain + hash keccak256 on-chain          |
| Dashboard               | HTML/CSS/JS thuần, 8 trang, cập nhật realtime mỗi 5s                |
| Version control         | Git/GitHub                                                          |

## 5. Kết quả thực nghiệm

### 5.1. Hội tụ của Global Model qua từng round

Chạy 5 round trên 4 node (seed dữ liệu = 4, tổng 1199 mẫu). Kết quả: accuracy tăng từ 30.58% (model khởi tạo ngẫu nhiên) lên 98.50%, F1 lớp bất thường từ 46.83% lên 97.52%, loss giảm từ 0.754 xuống 0.048 — toàn bộ mức tăng này đến từ FedAvg, không node nào gửi dữ liệu thô đi đâu cả.

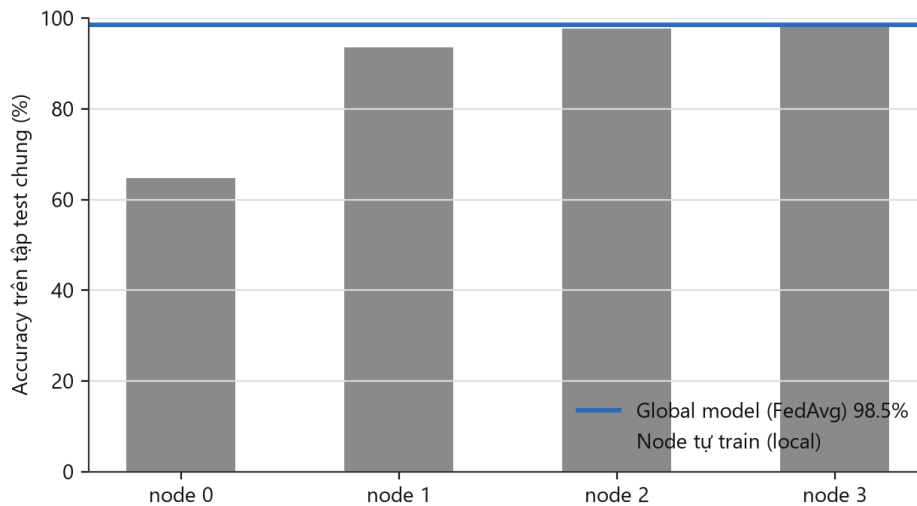
Hình 2. Độ hội tụ accuracy/F1 qua các round (nguồn: *results.json*)

| Round        | Accuracy | F1     | Loss   | Global model hash |
|--------------|----------|--------|--------|-------------------|
| 0 (khởi tạo) | 30.58%   | 46.83% | 0.7535 | —                 |
| 1            | 97.99%   | 96.67% | 0.0777 | c6024e6d5444f8... |
| 2            | 98.25%   | 97.10% | 0.0609 | 95ba6d01babaed... |
| 3            | 97.99%   | 96.67% | 0.0559 | d97fffb8cf1324... |
| 4            | 98.25%   | 97.12% | 0.0512 | 178c6453025dbf... |
| 5            | 98.50%   | 97.52% | 0.0479 | 8e355a5eb8fb43... |

Bảng 3. Chi tiết từng round, hash đọc trực tiếp từ smart contract

## 5.2. So sánh FedAvg với từng node tự huấn luyện riêng lẻ

Để chứng minh giá trị của Federated Learning (không chỉ là chạy cho có), mỗi local model (chỉ train trên dữ liệu của 1 node) được chấm trên cùng tập test chung với global model. Ở round cuối:



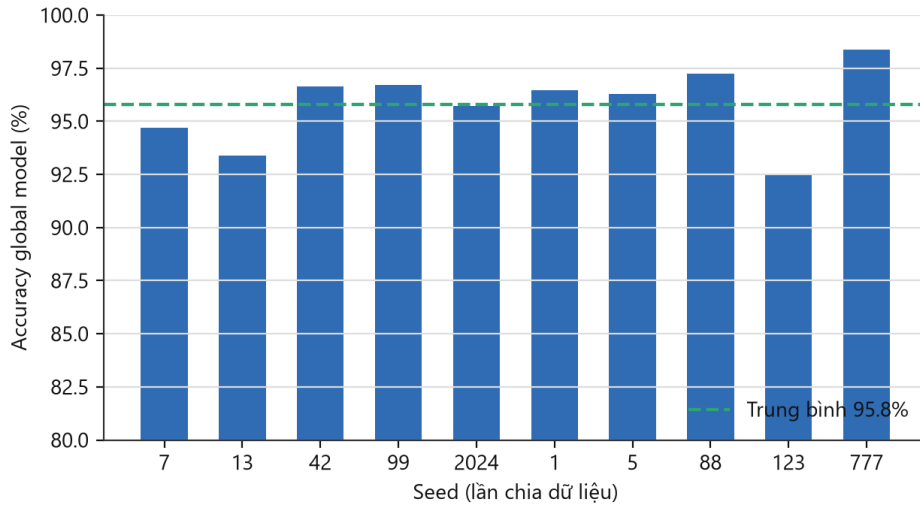
Hình 3. Global model (FedAvg) so với từng node tự train một mình

| Node   | Số mẫu | Accuracy tự train (trên tập test chung) | Accuracy global model trên dữ liệu riêng của node |
|--------|--------|-----------------------------------------|---------------------------------------------------|
| node 0 | 226    | 64.66%                                  | 90.27%                                            |
| node 1 | 368    | 93.48%                                  | 97.55%                                            |
| node 2 | 315    | 97.74%                                  | 100.00%                                           |
| node 3 | 290    | 98.25%                                  | 100.00%                                           |

Bảng 4. Global model đạt 98.50%, vượt trội mọi node tự train — node dữ liệu lệch nặng nhất (ít mẫu, tỉ lệ bất thường cao) tự train chỉ đạt độ chính xác thấp hơn hẳn.

## 5.3. Kiểm định độ ổn định qua nhiều lần chia dữ liệu

Một kết quả duy nhất trên một lần chia dữ liệu có thể chỉ là seed may mắn. Toàn bộ pipeline được chạy lại trên 10 seed khác nhau (mỗi seed = một bộ dữ liệu cảm biến hoàn toàn khác — tỉ lệ bất thường, số mẫu, giá trị cảm biến của mọi node đều đổi):



Hình 4. Accuracy của global model qua 10 lần chia dữ liệu khác nhau

| Chỉ số                                    | Giá trị             |
|-------------------------------------------|---------------------|
| Accuracy trung bình                       | 95.81% ± 1.8 điểm % |
| Accuracy thấp nhất / cao nhất             | 92.47% / 98.37%     |
| F1 lớp bất thường trung bình              | 94.16% ± 2.4 điểm % |
| FedAvg thắng mọi node tự train            | 10/10 lần           |
| Cách biệt trung bình so với node tốt nhất | +1.4 điểm %         |

Bảng 5. Tóm tắt độ ổn định (nguồn: results\_sweep.json, sinh bởi scripts/run\_sweep.py)

| Seed | Global (FedAvg) | Node tốt nhất tự train | Node kém nhất tự train | Cách biệt | Kết quả |
|------|-----------------|------------------------|------------------------|-----------|---------|
| 7    | 94.70%          | 94.35%                 | 62.19%                 | +0.4 %    | Thắng   |
| 13   | 93.40%          | 91.75%                 | 59.08%                 | +1.7 %    | Thắng   |
| 42   | 96.66%          | 94.22%                 | 71.43%                 | +2.4 %    | Thắng   |
| 99   | 96.73%          | 96.43%                 | 86.01%                 | +0.3 %    | Thắng   |
| 2024 | 95.73%          | 94.59%                 | 69.23%                 | +1.1 %    | Thắng   |
| 1    | 96.47%          | 92.16%                 | 49.02%                 | +4.3 %    | Thắng   |
| 5    | 96.31%          | 96.02%                 | 45.45%                 | +0.3 %    | Thắng   |
| 88   | 97.24%          | 96.13%                 | 64.64%                 | +1.1 %    | Thắng   |
| 123  | 92.47%          | 90.65%                 | 66.23%                 | +1.8 %    | Thắng   |
| 777  | 98.37%          | 98.04%                 | 72.55%                 | +0.3 %    | Thắng   |

Bảng 6. Chi tiết 10 lần chạy độc lập — FedAvg thắng mọi node tự train ở cả 10/10 lần

## 5.4. Kiểm thử smart contract

Bộ test Hardhat (test/aggregator.test.js) kiểm tra 6 kịch bản: đăng ký node và chặn đăng ký trùng, chặn node lạ gọi submitWeights, chặn nộp sai round/nộp 2 lần, chặn aggregate khi chưa đủ minNodes, luồng đầy đủ submit → aggregate → distributeReward → advanceRound (đối chiếu số token đúng công thức chia theo tỉ lệ mẫu), và chặn tài khoản không phải owner gọi các hàm quản trị. Toàn bộ 6/6 test pass.

## 5.5. Minh bạch và chống chối bỏ trên chuỗi

Dashboard (trang Round on-chain) tự động bấm lại file trọng số cục bộ ai\_model/weights/global\_round\_N.npy và so với globalModelHash đang lưu trên smart contract. Nếu khớp — hiển thị badge “Khớp” — nghĩa là file trọng số đang giữ đúng là file đã được chốt on-chain; chỉ cần sửa 1 byte trong file là badge lập tức chuyển sang “Lệch”, trong khi giá trị trên chuỗi thì không ai sửa được để che giấu, thể hiện đúng tính chất bất biến của blockchain.

## 6. Đánh giá rủi ro và phương án xử lý

| Rủi ro                                                        | Phương án xử lý                                                                                                                         |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Gas fee/độ trễ khi test nhiều lần trên Testnet                | Phát triển và test chủ yếu trên Hardhat local; chỉ deploy Sepolia ở giai đoạn cuối để demo                                              |
| Trọng số mô hình quá lớn để lưu on-chain                      | Lưu hash keccak256 (32 byte) on-chain, trọng số thật lưu off-chain (file .npy)                                                          |
| Mô hình FL không hội tụ do dữ liệu non-IID quá lệch           | Điều chỉnh mức Dirichlet $\alpha$ , tăng số round, dùng learning rate nhỏ; đã kiểm chứng hội tụ ổn định qua 10 seed khác nhau (mục 5.3) |
| Node gian lận gửi $\Delta w$ bất thường (poisoning)           | Hướng phát triển: kiểm tra độ lệch $\Delta w$ trước khi tổng hợp (chưa triển khai trong bản hiện tại)                                   |
| Một kết quả FL có thể chỉ là “ăn may” theo 1 lần chia dữ liệu | Chạy lại toàn bộ pipeline trên 10 seed độc lập (scripts/run_sweep.py) — FedAvg thắng mọi node tự train ở cả 10/10 lần                   |

## 7. Kết luận và hướng phát triển

Hệ thống BlockFL đã hiện thực hoá thành công luồng end-to-end IoT → Federated Learning → Blockchain: 4 edge node với dữ liệu non-IID huấn luyện cục bộ, FedAvg tổng hợp trọng số off-chain, smart contract ghi nhận minh bạch qua hash và tự động phân phối token thưởng theo đóng góp. Kết quả thực nghiệm cho thấy mô hình hội tụ ổn định (accuracy trung bình 95.8% qua 10 lần chia dữ liệu độc lập) và FedAvg luôn vượt trội so với việc từng node tự huấn luyện riêng lẻ.

Hướng phát triển tiếp theo:

- Cơ chế phát hiện node gian lận: kiểm tra độ lệch thống kê của  $\Delta w$  trước khi đưa vào aggregate().
- Deploy chính thức lên Testnet Sepolia kèm liên kết Etherscan cho từng giao dịch.
- Nhúng mô hình rút gọn (TensorFlow Lite) lên phần cứng thật (ESP32/Raspberry Pi) thay cho giả lập hoàn toàn bằng phần mềm.
- Lưu trữ trọng số trên IPFS thay vì file cục bộ để tăng tính phi tập trung.

## 8. Cấu trúc repository GitHub

Mã nguồn được tổ chức theo đúng cấu trúc thư mục quy định trong tài liệu hướng dẫn của học phần:

- README.md — hướng dẫn cài đặt, cấu hình môi trường và các bước chạy demo.
- contracts/ — BlockFLToken.sol, FederatedAggregator.sol.
- test/ — aggregator.test.js (6 test Hardhat).
- scripts/ — deploy.js (deploy contract), web3\_interface.py (cầu nối Web3.py), run\_sweep.py (kiểm định đa seed), build\_report.js (sinh báo cáo này).
- ai\_model/ — model.py, local\_train.py, fedavg.py và thư mục weights/ lưu trọng số theo round.
- iot\_code/ — edge\_node.py, data\_partition.py (giả lập & chia dữ liệu non-IID).
- dashboard/ — 8 trang giám sát thời gian thực (HTML/CSS/JS + Ethers.js).
- run\_demo.py — script điều phối chạy toàn bộ pipeline end-to-end.

Repository công khai (Public) trên GitHub, link: [Điền link GitHub repository].

## 9. Phân công nhiệm vụ

Đề cương ban đầu phân công 5 vai trò như sau (điền cụ thể tên/MSSV từng thành viên trước khi nộp):

| Vai trò                      | Nhiệm vụ chính                                                          |
|------------------------------|-------------------------------------------------------------------------|
| Blockchain Developer         | Viết/test/deploy smart contract, tích hợp token ERC-20, quản lý Testnet |
| AI/ML Engineer               | Xây dựng mô hình, cài đặt FedAvg, chia dữ liệu non-IID, đánh giá hội tụ |
| IoT/Backend Integrator       | Giả lập edge node, viết cầu nối Web3.py giữa AI và blockchain           |
| Dashboard/Frontend Developer | Xây dựng dashboard Web3, tích hợp MetaMask, hiển thị realtime           |
| PM & Báo cáo/Thuyết trình    | Quản lý tiến độ, tổng hợp báo cáo kỹ thuật, chuẩn bị demo               |

## 10. Tài liệu tham khảo

- McMahan, H. B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data.
- Tài liệu Solidity chính thức: <https://docs.soliditylang.org>
- Hardhat Documentation: <https://hardhat.org>
- Web3.py Documentation: <https://web3py.readthedocs.io>
- Ethers.js Documentation: <https://docs.ethers.org>